

Cmd

命令行

虽然图形界面能做的事情越来越多，但有很多高阶操作仍然需要使用命令行来解决。

本页面将简要介绍命令行的一些使用方法。

基础

Windows 自带的命令行界面有两个。「命令提示符」（cmd）是其中较为古老的一个，功能也相对简单。PowerShell 是较新的一个命令行界面，自带的功能丰富，但相对臃肿。两个界面都可以在开始菜单中找到。

类 Unix 系统（包含 macOS 和 Linux，以下称为 Unix）分为有图形界面和无图形界面两种情况。如果系统有图形界面（如使用 macOS 或者在 Linux 下安装了 GNOME、KDE 等图形界面），则命令行一般可以通过名为「终端」（Terminal 或 Console）的程序打开。没有图形界面的系统会在启动完成后自动进入命令行。

Windows 下的命令行长这样：

```
1 C:\Users\chtholly>
```

在命令行上输入的指令会显示在 > 以后。

```
1 C:\Users\chtholly>echo "Hello World!"
```

Unix 下的命令行长这样（以 Debian/Ubuntu 为例，其它系统的命令行大体类似）：

```
1 chtholly@seniorious:~$
```

在命令行上输入的指令会显示在 \$ 以后。

```
1 chtholly@seniorious:~$ echo "Hello World!"
```

如果在 Unix 下使用 root 登录命令行，那么 \$ 会被替换成 #：

```
1 root@seniorious:~# apt-get install gcc
```

命令行的 >, \$ 或 # 之前会显示一个路径，这个路径就是工作目录（working directory），或者当前目录。在 Unix 下当前目录有时会显示成类似 ~/folder 的形式，最开头的 ~ 就是当前登录的用户的主目录。用户 chtholly 的主目录在不同系统下的位置是不同的：在 Linux 下，其主目录位于 /home/chtholly，而在 macOS 下，其主目录位于 /Users/chtholly。

语法和常用命令¹

文件系统相关

先介绍文件系统里描述位置的两种方式，相对路径和绝对路径。

- 相对路径：用相对当前路径的位置关系来描述位置。例如当前路径为 ~/folder，则 ./a.cpp 实际上指的就是 ~/folder/a.cpp 这个文件。随着当前路径的变化，相对路径描述的位置也可能发生改变。
- 绝对路径：用完整的路径来描述位置。例如 ~/folder/a.cpp 就是一个绝对路径的例子。绝对路径描述的位置不随当前路径的变化而改变。

Windows/Unix 用 . 代表当前目录，.. 代表当前目录的父目录。特别地，在 Unix 下，用 ~ 表示用户主目录（注意 ~ 由 shell 展开，因此在其他地方可能不可用）。

在 Windows/Unix 下，使用 pwd 命令可以打印当前的目录，cd <目录> 命令可以切换当前的目录。例如，cd folder 会切换到当前目录的 folder 子目录；cd .. 会切换到当前目录的父目录。

在 Windows 下，使用 dir 命令可以列出当前目录的文件列表。在 Unix 下，列出文件列表的命令是 ls。特别的，在 PowerShell 下，可以使用与 Unix 相同的 ls 命令。

在 Windows 下, 使用 `md <目录>` 或者 `mkdir <目录>` 命令创建一个新目录, 使用 `rd <目录>` 命令删除一个目录。在 Unix 下, 这两个命令分别是 `mkdir` 和 `rmdir`。需要注意的是, 使用 `rd` 或是 `rmdir` 删除一个目录前, 这个目录必须是空的。如果想要删除非空目录 (和该目录下的所有文件) 的话, Unix 下可以执行 `rm -r <目录>` 命令, Windows 下可以执行 `rd /s <目录>` 命令。

重定向机制

我编译了一个程序, 它从标准输入读入, 并输出到标准输出。然而输入文件和输出文件都很大, 这时候能不能想办法把输入重定向到指定的输入文件, 输出重定向到指定的输出文件呢?

使用如下命令即可实现。

```
1 $ command < input > output
```

例如, `./prog < 1.in > 1.out` 这个命令就将让 `prog` 这个程序从当前目录下的 `1.in` 中读入数据, 并将程序输出覆盖写入到 `1.out`。

▼ Warning

`1.out` 原本的内容会被覆盖, 如果想要在原输出文件末尾追加写入, 请使用 `>>`, 即 `./prog >> 1.out` 的方式做输出重定向

注意, PowerShell 只支持输出重定向, 不支持输入重定向。

事实上, 大多数 OJ 都采用了这样的重定向机制。选手提交的程序采用标准输入输出, 通过重定向机制, 就可以让选手的程序从给定的输入文件读入数据, 输出到指定的输出文件, 再进行文件比较就可以评测了。

执行程序

对于一个可执行程序或是批处理脚本, 只需在命令行里直接输入它的文件名即可执行它。

当然, 执行一个文件时, 命令行并不会把所有目录下的文件都找一遍。环境变量 `PATH` 描述了命令行搜索路径的范围, 命令行会在 `PATH` 中的路径寻找目标文件。

对于 Windows 系统, 当前目录也在命令行的默认搜索范围内。例如 Windows 系统中, 输入 `hello` 命令就可以执行当前目录下的 `hello.exe`。但是在 PowerShell 中, PowerShell 默认不会从当前目录寻找可执行文件 (这与在 Unix 的行为一致), 因而在 PowerShell 中需要使用相对路径或绝对路径调用当前目录下的可执行文件, 例如 `.\hello.exe`, 否则, 你将看到以下报错:

```
1 PS> hello
2 hello: The term 'hello' is not recognized as a name of a cmdlet,
3 function, script file, or executable program.
4 Check the spelling of the name, or if a path was included, verify that
5 the path is correct and try again.
6
7 Suggestion [3,General]: The command hello was not found, but does exist
8 in the current location. PowerShell does not load commands from the
9 current location by default. If you trust this command, instead type:
10 ".\hello". See "get-help about_Command_Precedence" for more details.
```

在 Unix 系统中, 当前目录并不在命令行的默认搜索范围内, 所以执行当前目录下的 `hello` 程序的命令就变成了 `./hello`:

```
1 $ hello
2 hello: command not found
3 $ ./hello
4 Hello World!
```

总结

上面介绍的用法只是命令行命令的一小部分, 还有很多命令没有涉及到。在命令行里输入帮助命令 `help`, 可以查询所有基本命令以及它们的用途。

下面给出 Windows 系统和 Unix 系统的命令对照表, 以供参考。

分类	Windows 系统	Unix 系统
文件列表	<code>dir</code>	<code>ls</code>
切换目录	<code>cd</code>	<code>cd</code>
建立目录	<code>md</code>	<code>mkdir</code>
删除目录	<code>rd</code>	<code>rmdir</code>
比较文件	<code>fc</code>	<code>diff</code>

分类 **Windows 系统** **Unix 系统**

复制文件	<code>copy</code>	<code>cp</code>
移动文件	<code>move</code>	<code>mv</code>
文件改名	<code>ren</code>	<code>mv</code>
删除文件	<code>del</code>	<code>rm</code>

使用命令行编译/调试

命令行编译

手动编译

在命令行下输入 `g++ a.cpp` 就可以编译 `a.cpp` 这个文件了（Windows 系统需提前把编译器所在目录加入到 `PATH` 中）。

编译过程中可以加入一些编译选项：

- `-o <文件名>`：指定编译器输出可执行文件的文件名。
- `-g`：在编译时添加调试信息（使用 `gdb` 调试时需要）。
- `-Wall`：显示所有编译警告信息。
- `-O1`, `-O2`, `-O3`：对编译的程序进行优化，数字越大表示采用的优化手段越多（开启优化会影响使用 `gdb` 调试）。
- `-DDEBUG`：在编译时定义 `DEBUG` 符号（符号可以随意更换，例如 `-DONLINE_JUDGE` 定义了 `ONLINE_JUDGE` 符号）。
- `-UDEBUG`：在编译时取消定义 `DEBUG` 符号。
- `-lm`, `-lgmp`：链接某个库（此处是 `math` 和 `gmp`，具体使用的名字需查阅库文档，但一般与库名相同）。

▼ Note

在 Unix 下，如使用了标准 C 库里的 `math` 库（`math.h`），则需在编译时添加 `-lm` 参数。²

使用 GNU Make 的内置规则⁹

对于名为 `qwq.c/cpp/p` 的 C, C++, Pascal 程序源代码，可以使用 `make qwq` 自动编译成对应名为 `qwq` 的程序。

如需添加额外的编译选项，请使用 `export CFLAGS="xxx"` 或者 `export CPPFLAGS="xxx"` 定义。

Sanitizers

介绍

`sanitizers` 是一种集成于编译器中，用于调试 C/C++ 代码的工具，通过在编译过程中插入检查代码来检查代码运行时出现的内存访问越界、未定义行为等错误。

它分为以下几种：

- `AddressSanitizer`⁵：检测对堆、栈、全局变量的越界访问，无效的释放内存、内存泄漏（实验性）。
- `ThreadSanitizer`⁶：检测多线程的数据竞争。
- `MemorySanitizer`⁷：检测对未初始化内存的读取。
- `UndefinedBehaviorSanitizer`⁸：检测未定义行为。

使用方式

最新版本的 `clang++`、`g++` 以及 `MSVC`（部分支持）均已内置 `sanitizers`，但功能和使用方法有所不同，这里以 `clang++` 为例，它的使用方法如下：

```
1 $ clang++ -fsanitize=&LTname> test.cc
```

其中 `<name>` 即为要启用的功能（一个 `sanitizer` 可理解为一些功能的集合），例如：

```
1 $ clang++ -fsanitize=memory test.cc # 启用 MemorySanitizer
2 $ clang++ -fsanitize=signed-integer-overflow test.cc # 启用有符号整型溢出检测
```

之后直接像平常一样运行可执行文件即可，如果 `sanitizer` 检测到错误，则会输出到 `stderr` 流，例如：

```
1 $ ./a.out
2 test.cc:3:5: runtime error: signed integer overflow: 2147483647 + 1 cannot be represented in type 'int'
```

时间/内存代价

显而易见，这些调试工具会严重拖慢代码的运行时间和增大所用内存，以下为使用它们的时间/内存代价：

名称	所增大内存倍数	所增大时间倍数
AddressSanitizer	N/A	2
ThreadSanitizer	5~15	5~10
MemorySanitizer	N/A	3
UndefinedBehaviorSanitizer	N/A	N/A

命令行调试

在命令行下，最常用的调试工具是 gdb。

执行 `gdb a` 就可以调试 `a` 程序。

以下是几个 gdb 调试的常用命令（大多数命令可以缩写，用命令开头的若干个字母就可以代表该命令）：

- `list (l)`：列出程序源代码，如 `l main` 指定列出 `main` 函数附近的若干行代码。
- `break (b)`：设置断点，如 `b main` 表示在 `main` 函数处设置断点。
- `run (r)`：运行程序直到程序结束运行或遇到断点。
- `continue (c)`：在程序遇到断点后继续执行，直到程序结束运行或到达下一个断点。
- `next (n)`：执行当前行语句，如果当前行有函数调用，则将其视为一个整体执行。
- `step (s)`：执行当前行语句，如果当前行有函数调用，则进入该函数内部。
- `finish (fin)`：继续执行至当前函数返回。
- `call`：调用某个函数，例如：`call f(2)`（以参数 2 调用函数 `f`）。
- `quit (q)`：退出 gdb。
- `display (disp)`：指定程序暂停时显示的表达式。
- `print (p)`：打印表达式的值。

`display` 和 `print` 指令都支持控制输出格式，其方法是在命令后紧跟 `/` 与格式字符，例如 `p/d test`（按照十进制打印变量 `test` 的值），支持的格式字符有：

格式字符	对应格式
d	按十进制格式显示变量
x	按十六进制格式显示变量
a	按十六进制格式显示变量
t	按二进制格式显示变量
c	按字符格式显示变量
f	按浮点数格式显示变量
u	按十进制格式显示无符号整型
o	按八进制格式显示变量

命令行使用技巧

自动补全

补全是 Shell 提供的基本功能之一，主要用于减少命令行使用中的输入量和 `typo` 概率。

一般情况下，使用补全的快捷键一般是 `Tab`，按下后 Shell 会根据已输入的字符补全信息。

不同的 Shell 提供了能力不尽相同的补全能力。

以下是常见 Shell 的补全能力³：

Shell	补全能力（补全范围）
cmd (Windows 的传统控制台)	文件路径

Shell	补全能力（补全范围）
PowerShell	文件路径、PATH 中的命令名、内建命令名、函数名、命令参数，支持模糊匹配，自动纠错
Bash	文件路径、PATH 中的命令名、内建命令名、函数名、命令参数
Zsh	文件路径、PATH 中的命令名、内建命令名、函数名、命令参数，支持模糊匹配，自动纠错和建议
Fish	文件路径、PATH 中的命令名、内建命令名、函数名、命令参数，支持模糊匹配，补全时可显示参数功能，自动纠错和建议

▼ Note

PowerShell 的部分功能需要 PSReadline Module 载入或者位于 PowerShell ISE 中。
Bash 的补全功能一般需要一个名为 `bash-completions` 的包才能获得完整功能，部分软件的补全文件由软件包自带。
Zsh 完整的补全功能需要配合用户预定义的文件（一般随 Zsh 包或对应软件包安装）。
Fish 在默认配置下提供良好完整的补全功能，但仍有部分官方未覆盖到的软件的补全文件由软件自行提供。

帮助文档

一般来说，命令行下的程序都附有「帮助」，Windows 下一般使用 `command /?` 或者 `command -?` 获取，Unix-like（例如 Linux）上一般使用 `command --help` 或者 `command -h` 获取（但是 BSD 下的「帮助」往往过分简略而难以使用）。
此外，在 Unix-like 系统上，还有可通过 `man command` 获取的「手册」(manual)，相比「帮助」一般更为详细。

built-in time 和 GNU time

测试程序运行时间时，我们通常可以使用 `time` 命令。
但是这个命令实际上在系统中有两个对应的命令：一个是部分 Shell（例如 Bash）内建的命令，一个是 GNU time（是一个单独的软件）。这两个之间存在一些差异。
一般在 Bash 中直接使用 `time` 调用的是 Bash 内建的版本，我们可以使用 `TIMEFORMAT` 环境变量控制其输出格式，例如将其设为 `%31R` 即可输出三位精度的实际运行时间，`%31U` 即可输出三位精度的用户空间运行时间。⁴

如果想要调用 GNU 版本的 `time`，则需使用 `\time` 或者 `/usr/bin/time` 调用，但是它的输出格式并不易读，我们可以附加 `-p` 参数（即为 `\time -p`）来获得易读的输出。

管道

假设我们现在有两个程序 A 和 B，都用标准输入输出，如何让 A 的输出重定向到 B 的输入？
我们可以使用上文中提到的重定向的方式，先把 A 的输出重定向到一个临时文件，在把 B 的输入重定向到这个临时文件上。
但这个方法很低效，不仅需要创建新的文件，磁盘 IO 的操作也可能成为瓶颈，而且两个程序不能同时运行，必须等 A 跑完了才能开始跑 B。有没有更好的方法？
有，那就是 **管道**，使用起来也非常简单，如下操作即可：

```
1 $ A | B
```

这会在内存创建一个管道，然后两个程序被同时启动。程序 A 每次要输出被重定向到这个管道中，而这个管道本身不会存储数据（其实有一个很小的缓冲区）。在 B 读取之前，A 的输出操作会被阻塞，等到 B 把数据读入以后，A 的输出才能继续进行。这样优美地解决了上述的问题，没有磁盘 IO 操作，两份代码同时运行，也没有额外消耗很多的内存储存中间结果。

命名管道

有时候我们不只是要把一个程序的输出重定向到另一个的输入。比如在做 IO 交互题的时候，经常需要将 A 的输出重定向到 B 的输入，B 的输出重定向到 A 的输出，这个时候用上文提到的普通管道就无能为力了。而重定向到文件，有无法让两个程序同时运行。这个时候就需要一个长得像文件的管道——命名管道。
在 Unix 系统中，可以使用如下命令创建命名管道（以命名为 `my_pipe` 举例）：

```
1 $ mkfifo my_pipe
```


这个时候使用 `ls` 命令列出当前目录下的文件，会发现多了一个 `my_pipe` 的文件。这就创建了一个命名管道，文件名后的 `|` 代表这是一个管道文件。然后就可以像文件的重定向一样向这个管道中读写了。
通过命名管道，我们可以通过这样的方式让两个程序交互：

```
1 $ mkfifo input output
2 $ ./checker > input < output # 这里一定要把 > input 写在前面, 不然 shell 会先打开 output 管道, 而这个管道现在并没有东西, 会阻塞 checker 的运行。
3 $ ./my_code < input > output
```

使用完后, 可以像普通文件一样用 `rm` 命令删除命名管道。

参考资料与注释

1. 刘汝佳《算法竞赛入门经典（第2版）》附录A 开发环境与方法 [↩](#)
 2. [Why do you have to link the math library in C?↩](#)
 3. [Comparison_of_command_shells#Interactive_features↩](#)
 4. <https://unix.stackexchange.com/a/70655> [↩](#)
 5. <https://clang.llvm.org/docs/AddressSanitizer.html> [↩](#)
 6. <https://clang.llvm.org/docs/ThreadSanitizer.html> [↩](#)
 7. <https://clang.llvm.org/docs/MemorySanitizer.html> [↩](#)
 8. <https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html> [↩](#)
 9. [Catalogue of Built-In Rules↩](#)
-

本页面最近更新: 2023/8/20 23:10:26, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [CoelacanthusHex](#), [StudyingFather](#), [CoderOJ](#), [mcendu](#), [qinyihao](#), [ayalhw](#), [CCXXLI](#), [Early0v0](#), [Enter-tainer](#), [ksyx](#), [NachtgeistW](#), [ouuan](#), [swiftqwq](#), [Tiphereth-A](#), [Xeonacid](#), [xiaofu-15191](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用